



DataFrame / Matplotlib 다루기

DataFrame

- 데이터 확인 및 요약

- 1.1. `df.head(n)`
- 1.2. `df.tail(n)`
- 1.3. `df.info()`
- 1.4. `df.describe(...)`

- 인덱싱 및 선택

- 2.1. `df[col]`
- 2.2. `df[[col1, col2, ...]]`
- 2.3. `df.loc[row_indexer, col_indexer]`
- 2.4. `df.iloc[row_pos, col_pos]`
- 2.5. `df.at[label, col]`
- 2.6. `df.iat[i, j]`
- 2.7. `df[mask]`

- 결측치 처리

- 3.1. `df.isna()` / `df.isnull()`
- 3.2. `df.dropna(...)`
- 3.3. `df.fillna(...)`
- 3.4. `df.replace(...)`

- 컬럼 조작 및 변형

- 4.1. `df.rename(...)`
- 4.2. `df.drop(columns=...)`
- 4.3. `df.astype(...)`
- 4.4. `df.assign(...)`
- 4.5. `df.apply(...)`
- 4.6. `df.applymap(...)`
- 4.7. `df['col'].map(...)`

- 그룹화 및 피벗

- 5.1. `df.groupby(...).agg(...)`
- 5.2. `df.pivot_table(...)`

- 결합 및 병합

- 6.1. `pd.concat(...)`
- 6.2. `df.merge(...)`
- 6.3. `df.join(...)`

- 정렬 및 인덱스 관리

- 7.1. `df.sort_values(...)`

7.2. `df.sort_index(...)`

7.3. `df.reset_index(...)`

7.4. `df.set_index(...)`

7.5. `df.reindex(...)`

- **입출력(I/O)**

8.1. `pd.read_csv(...)`

8.2. `df.to_csv(...)`

8.3. `df.to_excel(...)`

8.4. `df.to_json(...)`

- **문자열 및 날짜/시간 처리**

9.1. `df['col'].str.contains(...)`

9.2. `df['col'].str.replace(...)`

9.3. `df['col'].dt.year / .dt.month / .dt.day / .dt.strftime(...)`

- **기타 유용 기능**

10.1. `df.duplicated(...)`

10.2. `df.drop_duplicates(...)`

10.3. `df.sample(...)`

10.4. `df.memory_usage(...)`

1. 데이터 확인 및 요약

```
import pandas as pd
```

```
# 예시 DataFrame 생성
```

```
df = pd.DataFrame({  
    'A': [10, 20, 30, 40, 50],  
    'B': ['x', 'y', 'z', 'x', 'y'],  
    'C': [1.5, None, 3.5, 4.5, None]  
}, index=['r0', 'r1', 'r2', 'r3', 'r4'])
```

	A	B	C
r0	10	x	1.5
r1	20	y	NaN
r2	30	z	3.5
r3	40	x	4.5
r4	50	y	NaN

- `df.head(n)`

- **기능:** DataFrame의 상위 n개 행을 보여줍니다.

- **작동 방식 및 결과:** 맨 위부터 0번째 행부터 n-1번째 행까지 잘라낸 DataFrame을 반환하며, 컬럼 구조는 원본과 동일합니다.

- **예제**

```
df.head(3)
```

	A	B	C
r0	10	x	1.5

```
r1  20  y  NaN
r2  30  z   3.5
```

- `df.tail(n)`

- 기능: DataFrame의 하위 n개 행을 보여줍니다.
- 작동 방식 및 결과: 맨 뒤에서부터 마지막 행까지 역순으로 n개를 추출한 DataFrame을 반환하며, 컬럼 구조는 원본과 동일합니다.
- 예제

```
df.tail(2)
```

```
   A  B    C
r3 40  x  4.5
r4 50  y  NaN
```

- `df.info()`

- 기능: 각 컬럼의 데이터 타입, 결측치 개수, 메모리 사용량을 요약 출력합니다.
- 작동 방식 및 결과: 화면에 텍스트로 요약 정보를 출력하고, 반환값은 없습니다.
- 예제

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, r0 to r4
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0    A      5 non-null      int64
1    B      5 non-null      object
2    C      3 non-null      float64
dtypes: float64(1), int64(1), object(1)
memory usage: 160.0+ bytes
```

- `df.describe(include=None, exclude=None)`

- 기능: 수치형(또는 지정한 타입) 컬럼들의 주요 통계량(개수·평균·표준편차·사분위수 등)을 계산합니다.
- 작동 방식 및 결과: 지정한 컬럼별로 통계 함수를 적용하여, 통계량이 행으로, 컬럼명이 열로 구성된 요약 DataFrame을 반환합니다.
- 예제

```
df.describe()
```

```
count      A      C
mean    30.000000  3.166667
std     15.811388  1.527525
min     10.000000  1.500000
25%     20.000000  2.500000
50%     30.000000  3.500000
```

```
75%    40.000000    4.000000
max     50.000000    4.500000
```

2. 인덱싱 및 선택

- `df[col]`

- 기능: 단일 컬럼을 `Series` 로 추출합니다.
- 작동 방식 및 결과: 해당 컬럼명에 해당하는 값들만 모아 `Series` 객체로 반환합니다.
- 예제

```
df['A']
```

```
r0    10
r1    20
r2    30
r3    40
r4    50
Name: A, dtype: int64
```

- `df[[col1, col2, ...]]`

- 기능: 여러 컬럼을 모아 새로운 DataFrame으로 추출합니다.
- 작동 방식 및 결과: 지정한 컬럼들만 포함하는 DataFrame을 반환합니다.
- 예제

```
df[['A', 'C']]
```

```
   A    C
r0  10  1.5
r1  20  NaN
r2  30  3.5
r3  40  4.5
r4  50  NaN
```

- `df.loc[row_indexer, col_indexer]`

- 기능: 라벨 기반(row, column 이름)으로 부분 DataFrame 또는 `Series` 를 선택합니다.
- 작동 방식 및 결과: 행과 열에 지정된 이름(리스트, 슬라이스, 불리언 조건 등)에 따라 교차된 영역을 잘라낸 결과를 반환합니다.
- 예제

```
df.loc['r1':'r3', ['B', 'C']]
```

```
   B    C
r1  y  NaN
r2  z  3.5
r3  x  4.5
```

- `df.iloc[row_pos, col_pos]`

- 기능: 정수 위치 기반(row, column 위치)으로 부분 DataFrame 또는 `Series` 를 선택합니다.

- 작동 방식 및 결과: 0부터 시작하는 인덱스 위치에 따라 슬라이스하거나 개별 요소를 반환합니다.

- 예제

```
df.iloc[1:4, 0:2]
```

	A	B
r1	20	y
r2	30	z
r3	40	x

- `df.at[label, col]`

- 기능: 라벨 기반으로 단일 스칼라 값을 빠르게 접근·수정합니다.
- 작동 방식 및 결과: 지정한 행 라벨과 컬럼 라벨에 해당하는 하나의 값을 반환하거나 할당합니다.

- 예제

```
df.at['r2', 'C']
```

```
3.5
```

- `df.iat[i, j]`

- 기능: 위치 기반으로 단일 스칼라 값을 빠르게 접근·수정합니다.
- 작동 방식 및 결과: i번째 행, j번째 열의 값을 반환하거나 할당합니다.

- 예제

```
df.iat[3,1]
```

```
'x'
```

- `df[mask]`

- 기능: 불리언 배열(mask)이 `True` 인 행만 필터링합니다.
- 작동 방식 및 결과: `mask` 와 같은 길이의 Boolean Series/배열에서 `True` 인 행들만 모아 새로운 DataFrame을 반환합니다.
- 예제

```
mask = df['C'] > 3.0
df[mask]
```

	A	B	C
r2	30	z	3.5
r3	40	x	4.5

3. 결측치 처리

- `df.isna()` / `df.isnull()`

- 기능: 각 원소가 결측치인지 여부를 Boolean DataFrame/Series로 반환합니다.
- 작동 방식 및 결과: 원본과 동일한 크기의 DataFrame/Series에 `True` (결측) 또는 `False` 값을 채워 반환합니다.
- 예제

```
df.isna()
```

	A	B	C
r0	False	False	False
r1	False	False	True
r2	False	False	False
r3	False	False	False
r4	False	False	True

- `df.dropna(axis=0/1, how='any'/'all', subset=[...])`

- 기능: 결측치가 있는 행(또는 열)을 제거합니다.
- 작동 방식 및 결과: `how='any'` 면 하나라도 결측이 있는 축을, `how='all'` 이면 모두 결측인 축을 삭제한 새로운 DataFrame을 반환합니다.
- 예제

```
df.dropna(subset=['C'], how='any')
```

	A	B	C
r0	10	x	1.5
r2	30	z	3.5
r3	40	x	4.5

- `df.fillna(value=None, method=None, axis=0/1)`

- 기능: 결측치를 지정값 또는 보간 방식으로 채웁니다.
- 작동 방식 및 결과: `value` 를 쓰거나 `ffill` / `bfill` 처럼 인접값을 복사해 결측치를 메운 DataFrame을 반환합니다.
- 예제

```
df.fillna({'C':0})
```

	A	B	C
r0	10	x	1.5
r1	20	y	0.0
r2	30	z	3.5
r3	40	x	4.5
r4	50	y	0.0

- `df.replace(to_replace, value, inplace=False)`

- 기능: 특정 값 또는 패턴을 다른 값으로 치환합니다.
- 작동 방식 및 결과: `to_replace` 에 해당하는 모든 원소를 `value` 로 바꾼 DataFrame을 반환합니다.
- 예제

```
df.replace({'B':'x'}, 'X')
```

	A	B	C
r0	10	X	1.5
r1	20	y	NaN
r2	30	z	3.5

```
r3 40  X  4.5
r4 50  y  NaN
```

4. 컬럼 조작 및 변형

- `df.rename(columns={'old':'new'}, inplace=False)`

- 기능: 컬럼명을 변경합니다.
- 작동 방식 및 결과: 지정한 매핑에 따라 컬럼 레이블을 바꾼 DataFrame을 반환합니다.
- 예제

```
df.rename(columns={'A':'Alpha', 'B':'Beta'})
```

```
      Alpha Beta    C
r0      10    x  1.5
r1      20    y  NaN
...
```

- `df.drop(columns=[...], axis=1, inplace=False)`

- 기능: 지정한 컬럼을 삭제합니다.
- 작동 방식 및 결과: 해당 컬럼을 제외한 나머지로 구성된 DataFrame을 반환합니다.
- 예제

```
df.drop(columns=['C'])
```

```
      A  B
r0  10  x
r1  20  y
...
```

- `df.astype(dtype={'col':'type', ...})`

- 기능: 컬럼별 데이터 타입을 변경합니다.
- 작동 방식 및 결과: 각 컬럼을 지정된 타입으로 변환한 DataFrame을 반환합니다.
- 예제

```
df.astype({'A':'float64'})
```

```
      A  B    C
r0 10.0  x  1.5
...
```

- `df.assign(new_col=lambda x: expr)`

- 기능: 새 컬럼을 추가하거나 기존 컬럼을 갱신합니다.
- 작동 방식 및 결과: 람다 식 또는 값으로 계산한 결과를 `new_col`로 추가한 DataFrame을 반환합니다.
- 예제

```
df.assign(D=lambda x: x['A']*2)
```

```

      A  B    C  D
r0 10  x  1.5 20
...
```

- `df.apply(func, axis=0/1)`

- 기능: 행(axis=1) 또는 열(axis=0) 단위로 사용자 함수를 적용합니다.
- 작동 방식 및 결과: 각 행(또는 열)에 `func` 를 실행한 반환값을 모아 `Series` 나 `DataFrame` 으로 반환합니다.
- 예제

```
df.apply(lambda row: row['A'] + (row['C'] or 0), axis=1)
```

```

r0 11.5
r1 20.0
r2 33.5
r3 44.5
r4 50.0
dtype: float64
```

- `df.applymap(func)`

- 기능: 모든 원소에 함수를 적용합니다.
- 작동 방식 및 결과: 각 원소별로 `func` 를 실행한 결과를 동일 구조의 `DataFrame`으로 반환합니다.
- 예제

```
df.applymap(lambda v: v*2 if isinstance(v,(int,float)) else v)
```

```

      A  B    C
r0 20  x  3.0
...
```

- `df['B'].map(func)`

- 기능: `Series`에만 사용 가능, 각 값에 함수를 적용하거나 매핑 딕셔너리를 사용합니다.
- 작동 방식 및 결과: 각 원소별로 `func` 또는 딕셔너리 매핑을 적용한 `Series`를 반환합니다.
- 예제

```
df['B'].map({'x':1, 'y':2, 'z':3})
```

```

r0 1
r1 2
r2 3
r3 1
r4 2
Name: B, dtype: int64
```

5. 그룹화 및 피벗

- `df.groupby(by=[...]).agg({'col1': 'sum', 'col2': 'mean'})`

- **기능:** 지정한 키로 그룹화 후, 컬럼별 집계를 수행합니다.
- **작동 방식 및 결과:** 각 그룹마다 합계·평균 등을 계산하여, 그룹 키를 인덱스로 갖는 집계 결과 DataFrame을 반환합니다.
- **예제**

```
df.groupby('B').agg({'A': 'sum', 'C': 'mean'})
```

	A	C
B		
x	50	3.000
y	70	NaN
z	30	3.500

- `df.pivot_table(values='val', index='idx', columns='col', aggfunc='mean', fill_value=None, margins=False)`

- **기능:** 피벗 테이블(집계+재구조화)을 생성합니다.
- **작동 방식 및 결과:** `index` 와 `columns` 키로 그룹화하여 `aggfunc` 를 적용한 뒤, 2차원 테이블 형태로 재배치된 DataFrame을 반환합니다.
- **예제**

```
df.pivot_table(values='A', index='B', columns='C', aggfunc='count', fill_value=0)
```

	C	1.5	3.5	4.5	NaN
B					
x	1	0	1	0	
y	0	0	0	2	
z	0	1	0	0	

6. 결합 및 병합

- `pd.concat([df1, df2, ...], axis=0/1, ignore_index=False)`

- **기능:** 축 방향으로 여러 DataFrame을 연결합니다.
- **작동 방식 및 결과:** 행(axis=0) 또는 열(axis=1) 방향으로 데이터를 이어붙인 새로운 DataFrame을 반환합니다.
- **예제**

```
pd.concat([df.head(2), df.tail(2)], axis=0)
```

	A	B	C
r0	10	x	1.5
r1	20	y	NaN
r3	40	x	4.5
r4	50	y	NaN

- `df.merge(right, how='inner'/'left'/'right'/'outer', on=..., left_on=..., right_on=..., suffixes=('_x', '_y'))`

- **기능:** SQL 스타일 조인으로 두 DataFrame을 병합합니다.
- **작동 방식 및 결과:** 조인 키를 기준으로 행을 매칭하여, 한쪽 또는 양쪽 데이터를 합친 결과 DataFrame을 반환합니다.
- **예제**

```
other = pd.DataFrame({'B':['x','y'],'D':[100,200]})
df.merge(other, on='B', how='left')
```

	A	B	C	D
r0	10	x	1.5	100.0
r1	20	y	NaN	200.0
r2	30	z	3.5	NaN
r3	40	x	4.5	100.0
r4	50	y	NaN	200.0

- `df.join(other, how='left'/'right'/'inner', on=None)`

- 기능: 인덱스(또는 지정한 컬럼) 기준으로 병합합니다.
- 작동 방식 및 결과: 기본적으로 인덱스를 기준으로 `other` 와 결합한 DataFrame을 반환합니다.
- 예

```
other2 = pd.Series([9,8,7,6,5], index=df.index, name='E')
df.join(other2)
```

	A	B	C	E
r0	10	x	1.5	9
r1	20	y	NaN	8
r2	30	z	3.5	7
r3	40	x	4.5	6
r4	50	y	NaN	5

7. 정렬 및 인덱스 관리

- `df.sort_values(by='col', ascending=True/False, inplace=False)`

- 기능: 지정한 컬럼 값을 기준으로 행을 정렬합니다.
- 작동 방식 및 결과: 오름차순/내림차순 옵션에 따라 행 순서를 재배치한 DataFrame을 반환합니다.
- 예제

```
df.sort_values(by='A', ascending=False)
```

	A	B	C
r4	50	y	NaN
r3	40	x	4.5
r2	30	z	3.5
r1	20	y	NaN
r0	10	x	1.5

- `df.sort_index(axis=0/1, ascending=True/False)`

- 기능: 행 인덱스(axis=0) 또는 컬럼 레이블(axis=1)을 정렬합니다.
- 작동 방식 및 결과: 인덱스(또는 컬럼명)를 알파벳·숫자 순으로 정렬한 DataFrame을 반환합니다.
- 예제

```
df.sort_index()
```

(원본이 이미 정렬되어 있으므로 변경 없음)

- `df.reset_index(drop=False, inplace=False)`
 - 기능: 기존 인덱스를 컬럼으로 옮기거나 버리고, 기본 정수 인덱스로 재설정합니다.
 - 작동 방식 및 결과: 새로운 0...n-1 정수 인덱스를 부여한 DataFrame을 반환하며, `drop=True` 로 하면 기존 인덱스 컬럼을 삭제합니다.
 - 예제

```
df.reset_index()
```

	index	A	B	C
0	r0	10	x	1.5
1	r1	20	y	NaN
2	r2	30	z	3.5
3	r3	40	x	4.5
4	r4	50	y	NaN

- `df.set_index(keys, drop=True, inplace=False)`
 - 기능: 지정 컬럼을 새로운 인덱스로 설정합니다.
 - 작동 방식 및 결과: 해당 컬럼의 값을 인덱스로 바꾼 DataFrame을 반환하며, `drop=True` 면 원본 컬럼은 삭제됩니다.
 - 예제

```
df.set_index('B')
```

	A	C
B		
x	10	1.5
y	20	NaN
z	30	3.5
x	40	4.5
y	50	NaN

- `df.reindex(index=new_index, columns=new_columns)`
 - 기능: 지정한 인덱스·컬럼 순서로 재배치하며, 없으면 결측치로 채웁니다.
 - 작동 방식 및 결과: 새로운 축 순서대로 데이터를 배치한 후, 존재하지 않는 위치는 NaN인 DataFrame을 반환합니다.
 - 예제

```
df.reindex(index=['r2','r0','r3'], columns=['C','A','B'])
```

	C	A	B
r2	3.5	30	z
r0	1.5	10	x
r3	4.5	40	x

8. 입출력(I/O)

- `pd.read_csv(filepath_or_buffer, sep=',', header=0, names=None, dtype=None, parse_dates=None, na_values=None)`

- 기능: CSV 파일을 DataFrame으로 불러옵니다.
- 작동 방식 및 결과: 파일을 한 줄씩 읽어 구분자 기준으로 분할·파싱하여 DataFrame을 생성 후 반환합니다.

- `df.to_csv(path_or_buf, sep=',', index=True/False)`

- 기능: DataFrame을 CSV 파일로 저장합니다.
- 작동 방식 및 결과: 각 행·열의 값을 문자열로 변환하여 지정 경로에 기록하고, 반환값은 없습니다.
- 예제

```
df.to_csv('out.csv', index=False)
# → 현재 디렉터리에 out.csv 파일 생성
```

- `df.to_excel(path, sheet_name='Sheet1', index=True/False)`

- 기능: DataFrame을 Excel 파일로 저장합니다.
- 작동 방식 및 결과: 지정한 시트 이름으로 데이터를 기록한 뒤 파일을 생성합니다.
- 예제

```
df.to_excel('out.xlsx', index=False)
# → out.xlsx 파일 생성
```

- `df.to_json(path_or_buf, orient='records')`

- 기능: DataFrame을 JSON 형식으로 직렬화합니다.
- 작동 방식 및 결과: 레코드 단위로 딕셔너리 리스트를 만들어 JSON 문자열 혹은 파일로 반환합니다.
- 예제

```
df.to_json('out.json', orient='records')
# → out.json 파일 생성
```

9. 문자열 및 날짜/시간 처리

- `df['col'].str.contains(pattern, case=True/False, na=False, regex=True)`

- 기능: 문자열에 패턴이 포함되었는지 검사합니다.
- 작동 방식 및 결과: 각 원소에 대해 패턴 검색 후, Boolean Series를 반환합니다.
- 예제

```
df['B'].str.contains('x', na=False)
```

```
r0    True
r1   False
r2   False
r3     True
r4   False
Name: B, dtype: bool
```

- `df['col'].str.replace(pat, repl, regex=True)`

- 기능: 문자열 패턴을 다른 문자열로 치환합니다.
- 작동 방식 및 결과: 각 원소별로 치환을 수행한 새로운 Series를 반환합니다.
- 예제

```
df['B'].str.replace('x','X', regex=False)
```

```
r0    X
r1    y
r2    z
r3    X
r4    y
Name: B, dtype: object
```

- `df['Date']= pd.to_datetime(df['Date'], format='%d/%m/%Y', errors='raise')`
- `df['col'].dt.year / .dt.month / .dt.day / .dt.strftime(fmt)`
 - 기능: 날짜/시간 타입 Series에서 연·월·일을 추출하거나 포맷 변환합니다.
 - 작동 방식 및 결과: 각 원소의 해당 속성을 추출한 정수 Series 또는 포맷된 문자열 Series를 반환합니다.
 - 예제
(날짜 컬럼 예시가 없으므로 생략)

10. 기타 유용 기능

- `df.duplicated(subset=[...], keep='first'/'last')`
 - 기능: 중복 행 여부를 Boolean Series로 표시합니다.
 - 작동 방식 및 결과: 지정한 기준으로 중복된 첫/마지막 인스턴스만 `False`, 나머지를 `True` 로 표시한 Series를 반환합니다.
 - 예제

```
df.duplicated(subset=['B'])
```

```
r0    False
r1    False
r2    False
r3     True
r4     True
dtype: bool
```

- `df.drop_duplicates(subset=[...], keep='first'/'last', inplace=False)`
 - 기능: 중복 행을 제거합니다.
 - 작동 방식 및 결과: 중복 판정 후 나머지 행을 삭제한 DataFrame을 반환합니다.
 - 예제

```
df.drop_duplicates(subset=['B'])
```

```
   A  B    C
r0 10  x  1.5
r1 20  y  NaN
r2 30  z  3.5
```

- `df.sample(n=None, frac=None, random_state=None)`

- **기능:** 무작위로 일부 행을 추출합니다.
- **작동 방식 및 결과:** 지정된 개수(n)나 비율(frac)만큼 랜덤 샘플을 뽑아 DataFrame으로 반환합니다.
- **예제**

```
df.sample(2, random_state=0)
```

	A	B	C
r2	30	z	3.5
r4	50	y	NaN

- `df.memory_usage(deep=True)`

- **기능:** 컬럼별 메모리 사용량을 Byte 단위로 반환합니다.
- **작동 방식 및 결과:** 각 컬럼의 저장 방식에 따라 사용 중인 메모리 크기를 산출한 Series를 반환합니다.
- **예제**

```
df.memory_usage(deep=True)
```

Index	136
A	40
B	214
C	40
dtype: int64	

Matplotlib

- 초기 설정

- 1.1. `import matplotlib.pyplot as plt`
- 1.2. `plt.style.use('ggplot')`
- 1.3. `plt.rcParams` 설정

- Figure 및 Axes 객체 생성

- 2.1. `plt.figure(figsize, dpi)`
- 2.2. `plt.subplots(nrows, ncols, sharex/sharey, figsize, dpi)`
- 2.3. `fig.add_subplot(R, C, index)`

- 기본 그래프 종류 및 사용 상황

- 3.1. 선 그래프 – `ax.plot(...)`
- 3.2. 산점도 – `ax.scatter(...)`
- 3.3. 막대그래프 – `ax.bar(...)`
- 3.4. 히스토그램 – `ax.hist(...)`
- 3.5. 박스플롯 – `ax.boxplot(...)`
- 3.6. 파이 차트 – `ax.pie(...)`
- 3.7. 이미지 표시 – `ax.imshow(...)`

- 레이블·타이틀·범례·그리드

- 4.1. `ax.set_title(...)`
- 4.2. `ax.set_xlabel(...)` / `ax.set_ylabel(...)`
- 4.3. `ax.legend(...)`
- 4.4. `ax.grid(...)`

- 눈금·틱·축 범위

- 5.1. `ax.set_xticks(...)` / `ax.set_xticklabels(...)`
- 5.2. `ax.set_xlim(...)` / `ax.set_ylim(...)`

- 레이아웃 조정 및 다축

- 6.1. `plt.tight_layout()`
- 6.2. `ax.twinx()` / `ax.twiny()`

- 출력 및 저장

- 7.1. `plt.show()`
- 7.2. `plt.savefig(...)`

1. 초기 설정

```
import matplotlib.pyplot as plt
```

- 기능: `pyplot` 인터페이스를 `plt` 라는 이름으로 불러옵니다.
- 결과: `plt` 네임스페이스에서 Figure-Axes 생성 및 그리기 함수를 사용할 수 있게 됩니다.
- 팁: 대부분의 예제와 공식 문서가 `plt` 를 기준으로 작성되어 있으므로, 항상 스크립트 맨 앞에 위치시킵니다.

```
plt.style.use('ggplot')
```

- 기능: 'ggplot', 'seaborn', 'bmh' 등 미리 정의된 스타일을 한 줄로 적용합니다.
- 결과: 이후 생성되는 모든 Figure에 색상 팔레트·격자·폰트 스타일 등이 자동으로 바뀝니다.
- 팁: 여러 그래프를 동일한 스타일로 통일하고 싶을 때 유용합니다.

```
plt.rcParams['figure.figsize'] = (8, 5)
plt.rcParams['font.size'] = 12
```

- 기능: rcParams를 통해 기본 Figure 크기와 전체 폰트 크기를 설정합니다.
- 결과: 이어지는 모든 Figure가 기본으로 (8인치×5인치, 해상도 100 DPI, 폰트 크기 12pt)로 생성됩니다.
- 팁: 발표용 슬라이드나 보고서에 맞춰 한 번에 전역 설정을 바꿀 때 편리합니다.

2. Figure 및 Axes 객체 생성

```
plt.figure(figsize=(8, 5), dpi=100)
```

- 기능: 단일 Figure(캔버스)를 생성합니다.
- 결과: 비어 있는 Figure 창이 (8×5인치, 100dpi) 크기로 활성화됩니다.
- 팁: 서브플롯이 필요 없고, 한 장짜리 그림만 그릴 때 간단히 사용합니다.

```
fig, ax = plt.subplots(nrows=2, ncols=2,
                       sharex=True, sharey=False,
                       figsize=(10, 8), dpi=120)
```

- 기능: 2×2 격자의 Axes 배열을 한 번에 생성합니다.
- 결과:
 - fig: 전체 Figure 객체
 - ax: 2×2 모양의 Axes 배열
 - 각 서브플롯이 지정 크기 (10×8인치, 120dpi)로 배치됩니다.
- 팁: 동일 x축 스케일 비교가 필요할 땐 sharex=True를, y축은 독립적이어야 할 땐 sharey=False를 지정합니다.

```
ax3 = fig.add_subplot(2, 2, 3)
```

- 기능: 기존 Figure의 2×2 그리드 중 3번째 위치에 Axes를 추가합니다.
- 결과: 3번째 칸에 새로운 Axes가 생성되어 ax3에 할당됩니다.
- 팁: 비정형 레이아웃(예: 1행+2행 조합)이나, 나중에 서브플롯을 추가해야 할 때 유연합니다.

3. 기본 그래프 종류 및 사용 상황

그래프 종류	함수 시그니처	사용 상황	결과	주요 팁
선 그래프	<code>ax.plot(x, y, label='label', linestyle='-', linewidth=2, marker='o')</code>	시간 흐름에 따른 추이 시각화 (예: 시계열 데이터, 트렌드 분석)	x, y 좌표를 연결한 선과 마커가 그려집니다.	linestyle, marker, linewidth로 선·마커 스타일을 조정하세요.
산점도	<code>ax.scatter(x, y, s=20, marker='x', alpha=0.7)</code>	두 변수 간 분포·상관 관계 파악 (예: 키-몸무게 분포, 클러스터 관찰)	각 (x, y) 좌표에 점이 찍힙니다.	s로 점 크기, alpha로 투명도를 지정해 과도한 마커 겹침을 완화할 수 있습니다.

막대그래프	<code>ax.bar(categories, values, width=0.6, bottom=0, align='center')</code>	범주형 데이터 비교 (예: 월별 매출, 그룹별 평균)	각 카테고리 위치에 높이가 <code>values</code> 인 막대가 세로로 표시됩니다.	누적막대를 그릴 땐 <code>bottom</code> 파라미터에 이전 막대 높이를 전달하세요.
히스토그램	<code>ax.hist(data, bins=10, density=False, cumulative=False)</code>	연속형 분포 파악 (예: 시험 점수, 고객 방문 빈도)	데이터 분포를 구간별 막대 높이로 나타냅니다.	<code>bins</code> 를 직접 조정하거나 리스트로 명시해 세밀한 분포를 시각화할 수 있습니다.
박스플롯	<code>ax.boxplot(data, vert=True, notch=False, patch_artist=True)</code>	데이터 분포 요약 및 이상치 확인 (예: 그룹별 점수 분포)	사분위수, 중앙값, 이상치를 박스-수염 형태로 표시합니다.	<code>notch=True</code> 로 중앙값 주변 모양을 강조하거나, <code>patch_artist</code> 로 박스 내부 색을 칠해 가독성을 높일 수 있습니다.
파이 차트	<code>ax.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)</code>	부분-전체 비율 강조 (예: 시장 점유율, 예산 분배)	원형 조각으로 각 요소의 비율이 시각화됩니다.	조각이 너무 많으면 읽기 어려우므로 5개 이하 사용을 권장하며, <code>startangle</code> 로 회전각을 조정할 수 있습니다.
이미지 표시	<code>ax.imshow(arr, cmap='viridis', vmin=0, vmax=1, aspect='equal')</code>	2D 배열(이미지·열지도) 시각화 (예: 픽셀 강도, 히트맵)	배열 값을 색상으로 매핑한 이미지가 Axes에 렌더링됩니다.	<code>cmap</code> 을 적절히 고르고, <code>vmin / vmax</code> 로 컬러 스케일 범위를 고정하면 비교 시 일관성을 유지할 수 있습니다.

4. 레이블·타이틀·범례·그리드

```
ax.set_title('그래프 제목', loc='center', pad=10)
```

- 기능: 그래프 상단에 제목을 표시합니다.
- 결과: Figure 위쪽 중앙에 '그래프 제목' 이 10pt 여백과 함께 나타납니다.

```
ax.set_xlabel('X축 레이블', fontsize=12, labelpad=6)
ax.set_ylabel('Y축 레이블', fontsize=12, labelpad=6)
```

- 기능: 각 축 아래·옆에 텍스트를 표시합니다.
- 결과: 지정한 크기와 여백으로 레이블이 그려집니다.

```
ax.legend(loc='upper right', ncol=1, title='범례')
```

- 기능: `label=` 이 지정된 플롯 요소들의 범례 상자를 표시합니다.
- 결과: 우측 상단에 '범례' 제목과 함께 아이템 목록이 나타납니다.

```
ax.grid(True, which='major', axis='both', linestyle='--', alpha=0.5)
```

- 기능: 주요 눈금에 맞춰 격자를 그립니다.
- 결과: 점선(`--`)과 반투명(`alpha=0.5`) 효과가 적용된 격자가 배경에 깔립니다.

5. 눈금·틱·축 범위

```
ax.set_xticks([0,1,2,3])
ax.set_xticklabels(['A','B','C','D'], rotation=45, fontsize=10)
```

- 기능: x축 눈금 위치와 레이블을 직접 지정합니다.
- 결과: 0~3 위치에 A, B, C, D 가 45도 회전된 채 표시됩니다.

```
ax.set_xlim(0, 5)
ax.set_ylim(-1, 10)
```

- **기능:** 축의 표시 구간을 강제로 잘라냅니다.
- **결과:** 지정한 구간만큼 축을 잘라서 확대·축소된 상태로 보여줍니다.

6. 레이아웃 조정 및 다축

```
plt.tight_layout()
```

- **기능:** 서브플롯 간 충돌 여백을 자동으로 최적화합니다.
- **결과:** 레이블·타이틀·축 레이블이 겹치지 않게 적절히 배치됩니다.

```
ax2 = ax.twinx()
```

- **기능:** 동일한 x축을 공유하되, 오른쪽에 두 번째 y축을 추가합니다.
- **결과:** 주 y축과 별개로 `ax2` 에서 다른 척도로 값이 표시됩니다. (온도·습도 동시 시각화 등)

7. 출력 및 저장

```
plt.show()
```

- **기능:** 작성한 모든 Figure를 화면에 렌더링합니다.
- **결과:** GUI 창 또는 노트북 셀에 그래프가 표시됩니다.

```
plt.savefig('figure.png',
            dpi=300,
            format='png',
            bbox_inches='tight',
            transparent=False)
```

- **기능:** 현재 Figure를 이미지 파일로 저장합니다.
- **결과:** `figure.png` 파일이 생성되며, 해상도 300dpi, 여백 최소화(`bbox_inches='tight'`) 옵션이 적용됩니다.
- **팁:** 투명 배경이 필요하면 `transparent=True` 옵션을 추가하세요.